



1. Документация должна объяснять систему, а не пересказывать код

Когда я использую AI для генерации документации, моя задача не состоит в том, чтобы получить текстовое описание каждой строки программы. Такая документация быстро становится бесполезной, потому что разработчик и так может открыть исходный файл.

Хорошая документация должна отвечать на вопросы, которые нельзя быстро получить простым чтением одной функции. Мне важно понять назначение модуля, его публичный интерфейс, ограничения, зависимости, типичный сценарий использования и условия, при которых компонент возвращает ошибку.

Поэтому при работе с Hermes я буду просить агента сначала исследовать модуль, а уже потом писать документацию. Hermes располагает файловыми инструментами для чтения, поиска и изменения файлов проекта, поэтому агент может работать непосредственно с существующим репозиторием, а не только с фрагментом кода, вручную вставленным в чат.

2. Источником документации должны быть проверяемые данные проекта

При автоматической генерации есть важный риск: модель способна написать правдоподобный текст о функции, которой на самом деле нет.

Поэтому я заранее определяю источники фактов. Для API это фактические функции, классы, параметры и возвращаемые значения. Для поведения модуля важны тесты и требования. Для установки проекта — реальные зависимости и конфигурационные файлы. Для истории изменений — Git.

Если в коде отсутствует информация о каком-либо бизнес-правиле, я не хочу, чтобы Hermes придумывал это правило для красивого README. В таком случае корректнее явно указать, что информация не определена.

Таким образом, AI здесь выполняет роль технического автора, который собирает и структурирует существующие сведения, но не должен незаметно создавать новые требования.

3. README отвечает на вопрос, как начать работать с модулем

README я рассматриваю как точку входа в проект или отдельный компонент.

Разработчик, который впервые открыл каталог, должен достаточно быстро понять, для чего существует модуль, какие основные возможности он предоставляет и как проверить, что всё работает.

Для нашего `testing_lab` хороший README не должен превращаться в учебник по Python. Ему нужно объяснить назначение модуля возвратов, основные публичные компоненты, требования к окружению, способ запуска тестов и важные ограничения поведения.

Hermes особенно удобен здесь тем, что может перед генерацией README исследовать структуру файлов через `search_files`, прочитать нужные исходники через `read_file` и проверить существующие команды в самом проекте. Это уменьшает вероятность появления инструкции, которая выглядит убедительно, но фактически не запускается.

4. API-документация фиксирует публичный контракт

Когда я документирую API модуля, я прежде всего отделяю публичный интерфейс от внутренних деталей реализации.

Для функции мне важны её назначение, параметры, типы данных, возвращаемое значение, возможные исключения и существенные ограничения. Для класса дополнительно важна ответственность самого объекта и то, какие методы предназначены для использования другими компонентами.

Например, документация `RefundService.create_request()` должна объяснить условия создания заявки и результат операции. Но ей не обязательно подробно пересказывать каждую локальную переменную внутри метода.

AI позволяет быстро получить первый вариант такой документации, особенно если типы уже указаны в сигнатурах. Но после генерации необходимо сверить описание с кодом и тестами. Если документация утверждает, что метод возвращает `None`, а фактически возвращается словарь, это уже дефект документации.

5. Пояснение логики отличается от документации интерфейса

Иногда одного описания API недостаточно. Мне нужно объяснить, почему система принимает конкретное решение.

Например, документация интерфейса может сообщать, что `calculate_refund()` возвращает сумму возврата. Пояснение логики должно дополнительно раскрыть, как учитывается количество товара, как применяется сохранённая скидка и почему используется цена из заказа, а не текущая цена товара.

Я стараюсь не превращать такое объяснение в построчный пересказ программы. Вместо этого выделяю причинно-следственную связь между входными данными и результатом.

Hermes может исследовать связанные функции и тесты, прежде чем сформировать объяснение. Поэтому агент способен восстановить логику не только из одной функции, но и из того, как компонент используется в проекте.

6. Changelog описывает изменение продукта, а не содержимое commit

Changelog выполняет другую задачу. README объясняет текущее состояние проекта, а changelog фиксирует, что изменилось между версиями или этапами разработки.

Я не хочу просто копировать туда сообщения Git commit. Commit может называться `fix tests`, но пользователю или другой команде такое сообщение почти ничего не объясняет.

Хорошая запись changelog должна раскрывать наблюдаемое изменение. Например, если мы исправили граничное условие возврата, полезная запись сообщает, что теперь возврат разрешён ровно на четырнадцатый день после получения заказа.

Здесь Hermes может использовать Git как источник данных: анализировать diff и историю изменений, а затем преобразовывать технические изменения в понятное описание. Но итоговый текст необходимо соотносить с реальным diff, чтобы в changelog не появились функции, которых commit фактически не добавлял.

7. Onboarding-материалы должны уменьшать время входа в проект

Onboarding-документация предназначена для разработчика, который впервые начинает работать с репозиторием.

Здесь недостаточно написать общее описание архитектуры. Человеку нужно понять, как подготовить окружение, какие каталоги наиболее важны, откуда начинать чтение кода, как запустить тесты и какие правила проекта необходимо учитывать перед внесением изменений.

AI особенно полезен для первого формирования такого документа, потому что вручную автор проекта часто пропускает очевидные для себя шаги. Для нового разработчика они совсем не очевидны.

При этом onboarding должен проверяться практически. Если документ говорит выполнить определённую команду, я хочу убедиться, что она действительно существует и работает в текущей версии проекта.

8. Автоматическая документация может устареть сразу после изменения кода

Одна из основных проблем документации — рассинхронизация с программой.

Представим, что мы изменили сигнатуру функции, а README остался прежним. Код работает, тесты проходят, но новый разработчик получает неправильную инструкцию.

AI не решает эту проблему автоматически. Он только позволяет быстрее обновлять документацию после изменения кода.

Поэтому хороший агентный процесс должен сравнивать документацию с текущим состоянием репозитория. При изменении публичного интерфейса Hermes можно попросить найти связанные README, API-описания и onboarding-файлы, которые теперь требуют обновления.

Так документация становится частью разработки, а не отдельным текстом, который создаётся один раз и постепенно теряет связь с системой.

9. Hermes можно использовать как повторяемого технического автора

Если команда регулярно создаёт документацию одинаковым способом, этот процесс можно формализовать.

В Hermes Skills предназначены для хранения повторяемых процедур, включая последовательность действий, команды, шаблоны и требования к результату. Поэтому при необходимости мы можем создать собственный Skill, который будет определять правила генерации README или API-документации для наших проектов.

Кроме того, у Hermes существует отдельный optional Skill `code-wiki`, предназначенный для генерации документации по кодовой базе. Он работает с локальными репозиториями и использует существующие файловые и терминальные инструменты Hermes без обязательного внешнего сервиса.

В нашей следующей практике полноценную wiki строить не потребуется. Нам важнее научиться управлять источниками и областью генерации, поэтому мы ограничимся новым модулем, его API и README.

10. Сгенерированная документация тоже должна проходить проверку

Документацию нельзя считать правильной только потому, что она хорошо написана.

После генерации я проверяю конкретные утверждения: существуют ли указанные файлы, совпадают ли названия функций, корректны ли параметры, действительно ли команда запуска тестов работает и соответствует ли описанное поведение тестам и контракту.

При работе с Hermes это можно превратить в отдельный этап. Сначала агент исследует репозиторий и генерирует документацию через файловые инструменты. После этого он повторно читает созданные файлы, сопоставляет содержащиеся в них технические утверждения с исходным кодом и при необходимости запускает команды, указанные в README. Hermes предоставляет для этого файловые и терминальные toolsets.

В следующей практике мы продолжим работу с уже созданным `testing_lab`. Hermes сначала исследует `refunds.py`, `CONTRACT.md` и созданные тесты, затем сформирует API-документацию и README нового модуля. После генерации мы отдельно проверим, что документация действительно соответствует коду, а приведённые в ней команды можно выполнить в текущем проекте.